



Desarrollo de Aplicaciones Web I

T51

Jonie Esteban Miralda Cruz

13/12/2025

Proyecto Grupo 2

Sistema de Punto de Venta para Veterinaria

Integrantes:

- Fernanda Issabela Madrid Garcia
- Kevin Aaron Irias Ruiz
- Valeska Estefany Ventura Reyes

Contenido

Introducción.....	3
Principios de Programación Aplicados.....	4
Separación de Capas en Express.....	4
Validaciones.....	4
Manejo de Errores.....	4
Seguridad.....	5
Uso de JWT.....	5
Flujo de Git Flow Aplicado.....	6
Complicaciones y Soluciones.....	7

Introducción

El presente documento describe el desarrollo del Sistema de Punto de Venta para Veterinaria, un proyecto académico que tiene como objetivo principal la aplicación práctica de los fundamentos del desarrollo web moderno. El sistema está diseñado para automatizar y gestionar de manera eficiente los procesos básicos de una clínica veterinaria, centrándose en tres funcionalidades clave: la autenticación segura de usuarios, el registro de ventas de productos y servicios, y la generación de facturas para los clientes.

Para lograr esto, se ha implementado una arquitectura de cliente-servidor que separa claramente las responsabilidades. El backend, desarrollado con Node.js y el framework Express, se encarga de toda la lógica de negocio, la gestión de datos y la seguridad. Por su parte, el frontend, construido con Angular, proporciona una interfaz de usuario mínima pero funcional, enfocada en tres pantallas esenciales (login, venta y factura), que se comunican con el servidor a través de API REST. Los datos se persisten en una base de datos relacional MySQL, modelada para representar entidades como Usuarios, Productos, Clientes y Ventas.

Un pilar fundamental del proyecto es la aplicación rigurosa de buenas prácticas de programación y principios de ingeniería de software. Esto se evidencia en la separación de capas dentro del backend, donde módulos específicos (rutas, controladores, middlewares, configuración) aseguran un código organizado, mantenible y escalable. La seguridad se implementó a través de JSON Web Tokens (JWT) para la autenticación y autorización, complementada con el hashéo de contraseñas mediante bcrypt y el uso de variables de entorno para proteger información sensible.

Además, se estableció un manejo centralizado de errores mediante códigos de estado HTTP apropiados (**400**, **401**, **500**), lo que permite respuestas claras y consistentes ante fallos. Todo el ciclo de desarrollo se gestionó bajo el flujo de trabajo Git Flow, fomentando el trabajo colaborativo y ordenado en equipo. En conjunto, este proyecto no solo cumple con los requisitos funcionales de un punto de venta básico, sino que también sirve como un ejercicio integral para dominar el desarrollo de APIs robustas, seguras y bien estructuradas.

Principios de Programación Aplicados

Separación de Capas en Express

Se hace en base a las capas del Frontend y Backend. El Frontend está conformado por una aplicación de Angular, que organiza por características en una carpeta src. Tiene sus componentes que son el login, usuarios, ventas y facturación, y los servicios que se encargan de consumir las APIs. El Frontend se encarga de la interfaz y las llamadas HTTP, sin incluir lógica de negocio.

La capa del Backend consiste en uso de API REST con Express y módulos que se dividen las responsabilidades, por ejemplo:

- Routes: Se encarga de todas las rutas correspondientes al proyecto.
- Controllers: Tienen la lógica de cada una de las rutas.
- Config: Se encarga de la conexión global a la base de datos.
- Middlewares: Son toda esa lógica de autenticación y manejo de errores.

Se usan dos middlewares globales, que son el de `express.json` para poder parsear la información y el de `CORS` para poder hacer la conexión entre el frontend y el backend.

Validaciones

Se manejan dos tipos de validaciones. La primera es a nivel de controlador. Cada controlador cuenta con su validación, ya sean campos requeridos o credenciales incorrectas. Luego, se encarga de retornar un mensaje ya sea de éxito, de error, no autorizado, credenciales inválidas, etc. También se manejan validaciones a nivel de middleware, éste último se encarga de proteger las rutas de ventas, clientes, usuarios y productos por medio de un token. En caso de no estar el token presente, no haber autorización, superar el límite de tiempo (una hora), el token ha expirado o ya no es válido, responderá con un código de estado (401).

Manejo de Errores

Los errores fueron manejados por medio de códigos HTTP donde cada código indica el tipo de error al haber sido hecha la solicitud de manera incorrecta. Por ejemplo, el 400 que indica que faltan datos requeridos, el 401 para la falla en la autorización y el 500 cuando es una falla en la consulta, un error del servidor o en la base de datos.

El manejo de errores se puede ver en los controladores. Al enviar el objeto JSON tienen su respectivo estado y mensaje para manejar cada tipo de error.

Seguridad

Se usa JSON Web Token para proteger las rutas. Esto se ve en el middleware de autenticación. El middleware se encarga de verificar la autorización por medio de un bearer token. En caso de no ir el token, deniega el acceso a la ruta protegida. También, al momento de registrar usuarios las contraseñas son hasheadas por medio de la librería bcrypt; así se evitan vulnerabilidades en la base de datos al tener las contraseñas almacenadas de una manera cifrada. Se hace uso de variables de entorno por medio de la librería dotenv para manejar credenciales de la base de datos. Esto ayuda a que todos los datos confidenciales se manejen en un archivo que no es compartido, solo visto por los desarrolladores.

Uso de JWT

El proyecto hace uso de JSON Web Token como una forma de autenticar y autorizar a los usuarios. Todo esto se divide en 3 componentes. Primero, la generación del token. Al momento de iniciar sesión, el aplicativo web valida las credenciales del usuario y las compara con el correo y la contraseña hasheada en la base de datos. En caso de ser correctas, se genera un token que contiene tanto el id y correo del usuario, con una expiración de una hora. Segundo, se usa un middleware de autenticación. Este middleware se encarga de verificar el token en las rutas protegidas. En caso de ir el token, da acceso a la ruta y en caso de que no haya un token, deniega el acceso. Como tercero, están las rutas protegidas. El middleware de autenticación está solamente en las rutas protegidas. Esto permite que los usuarios autenticados tengan acceso a las rutas. Solamente las rutas de registro y login permanecen públicas y pueden ser accedidas sin necesidad de un token.

Flujo de Git Flow Aplicado

1. main (rama principal - producción)



2. develop (rama de desarrollo)



3. features (ramas de características creadas):

Backend - Configuración y Seguridad:

- Feature-cofig-db
- feature/middleware-auth

Backend - Autenticación:

- feature/registro
- feature/login

Backend - Rutas:

- feature/routes-usuarios
- feature/routes-productos
- feature/routes-clientes
- feature/routes-ventas
- feature/routes-detalle-venta

Backend - Controladores:

- feature/controller-clientes
- feature/controller-ventas
- feature/controller-usuarios
- feature/controller-login
- feature/controller-registro
- feature/contoller-productos
- feature/contoller-detalle-venta

Frontend - Componentes:

- feature/pantalla-login
- feature/pantalla-venta
- feature/pantalla-factura

Correcciones (Problemas durante el desarrollo):

- revert-10-feature/controller-clientes
- revert-11-revert-10-feature/controller-clientes

↓

4. Merge de features → develop

↓

5. release (desde develop)

↓

6. Merge release → main

Complicaciones y Soluciones

Durante el desarrollo del proyecto se presentaron diversas complicaciones a nivel técnico, las cuales fueron abordadas y resueltas de manera progresiva, permitiendo un aprendizaje significativo.

Una de las principales dificultades fue el manejo de Git y Git Flow en un entorno colaborativo. Se presentaron conflictos al cambiar entre ramas, especialmente al pasar de develop a ramas feature, así como conflictos por archivos modificados simultáneamente o por cambios en los nombres de archivos. En una ocasión, incluso se eliminaron cambios de la rama develop y se realizó un *push* accidental a main. Como solución, el equipo aprendió a utilizar comandos como `git revert`, a resolver conflictos manualmente y a prestar mayor atención al flujo correcto de ramas antes de realizar *push* al repositorio.

Otra complicación importante fue la integración de las APIs, ya que algunas rutas devolvían errores debido a inconsistencias entre el backend y la base de datos, o por cambios realizados en la estructura del proyecto. Estos problemas se resolvieron revisando las consultas SQL, validando los datos enviados desde el frontend y asegurando que los controladores y rutas coincidieran con el modelo de base de datos definido.

En el desarrollo del frontend surgieron dificultades relacionadas con Angular, especialmente en la conexión con el backend. La plantilla inicial del frontend estaba configurada para consumir otro backend, lo que generó confusión al momento de integrar las nuevas APIs del proyecto. La solución fue crear servicios específicos para el módulo de ventas, desacoplando esa funcionalidad del resto de la plantilla y permitiendo que Angular consumiera correctamente las APIs desarrolladas en Node.js y Express.

Estas complicaciones permitieron fortalecer las habilidades técnicas del equipo en control de versiones, desarrollo backend y frontend, así como mejorar la forma de trabajo colaborativo.